Open Access

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Content-Addressed Caching for Reactive Notebooks

Dylan Madisetti[1], Myles Scolnick[1], and Akshay Agrawal[1]

[1]marimo

Abstract

We describe the caching mechanism behind resumable sessions in marimo, a reactive Python notebook. Cache keys are derived recursively from the reactive DAG: references are content-addressed where their bytes are reachable, and the producing cell's key is substituted where they are not, a Merkle-style recurrence that an edit invalidates at exactly the subtree it dominates. Keys are computed over compiled bytecode rather than source text, and the notebook that builds this paper re-derives the resulting invariances — to comments, formatting, cell reordering, and cell identifiers — against the implementation on every render. The same keys make results durable and portable: cached values cross process and session boundaries through a lazy stub mechanism and ship inside marimo's static WASM/HTML export, so a reader's browser rehydrates results — even values produced by libraries the browser cannot import — with no local Python installation. Microbenchmarks against representative baselines validate that the overhead stays interactive-class: hits on exploratory-scale payloads remain under the 100 ms threshold, and measuring the write path yields a break-even rule for when caching pays. The mechanism stays native to the reactive notebook and requires no per-call-site opt-in.

Keywords reactive notebooks, content-addressed caching, memoization, reproducibility, dataflow, marimo, Python

1. Introduction

Notebooks underpin scientific Python, but most of them do not survive a clean rerun. In a survey of 1.4 million public Jupyter notebooks, only about a quarter re-execute top-to-bottom without raising [1]. The cells a reader sees record what the author once ran, not what the notebook does now; out-of-order execution and hidden state mean reproducing a result requires replaying an execution order the notebook does not record.

Reactive notebooks mostly address this gap by treating cells, a unit of source code, as nodes in a dataflow graph. From the variables a cell reads (its *references*, `refs`) and the variables it binds (its *definitions*, `defs`), execution order is determined by data dependencies rather than source order, and editing a cell re-executes its dependents. Because the relation is derived statically rather than from a runtime trace, hidden state is hard to sustain and out-of-order execution is impossible at the cell level. Notable reactive notebooks include Pluto.jl [2], Observable [3], and Livebook [4], while IPyflow and nbsafety [5], [6] retrofit something close onto Jupyter through runtime dependency inference and program slicing. The lineage descends from a longer tradition of direct-manipulation programming environments [7], in which editing the source is itself the act that updates the running program. marimo [8] reinvents the reactive notebook for Python and is the focus of this paper.

Reactivity has a cost: where Jupyter lets a user re-evaluate just enough cells to repopulate memory, a dataflow pipeline reruns from its roots, so every session pays for every expensive

Published Jun 11, 2026

Correspondence to
Dylan Madisetti
dylan@marimo.io

cell. Yet a reactive notebook already knows what each cell depends on, and recomputation can be skipped wherever the cell body is deterministic.

Caching for notebooks and Python is not new. IncPy modifies the CPython interpreter to memoize function calls automatically [9]; knitr caches literate-document chunks with hand-declared dependency chains [10]; jupyter-cache re-executes a notebook wholesale when any code cell changes [11]. `mandala` memoizes decorated calls inside a `with storage:` block [12], Kishu checkpoints whole sessions for time travel [13], ElasticNotebook migrates live state across machines [14], and diskcache provides a byte-keyed store at the bottom of the stack [15]. Each asks the user to opt in at a different boundary. Starting from marimo's reactive DAG produces a cache the user does not have to opt into per call site, because the same parse pass that derives a cell's references also derives the inputs to its key — knitr's hand-maintained dependency declarations arrive for free.

Concretely we target three properties. (a) Skip expensive recomputation when references and source are unchanged. (b) Preserve reactive determinism within a session under reordering and partial reruns. (c) Make cached artifacts transportable through marimo's static WASM/HTML export. Out of scope are full session restoration in the sense of [13], distributed execution, and reproducibility of arbitrary Python notebooks [1]; we claim deterministic reuse within a reactive session, a strictly smaller and more defensible target.

2. Background and Related Work

Michie's memo functions [16] are the canonical outline of "function caching": skip recomputation when an input-derived key matches a stored key. Caching a *cell* forces the question of what a cell's "inputs" are. A reactive notebook's static derivation gives a graph that is stable across runs, so the key must be a function of the graph the source defines — not of a particular execution trace — together with the values bound at evaluation time. Cell-level dataflow tracking has an earlier antecedent in [17], and Rex [18] probes the boundaries of reactivity in marimo specifically.

To build a key from a cell's refs, we look to build systems. Build Systems à la Carte [19] decomposes a build system into a *rebuilder* (deciding when to rerun) and a *scheduler* (deciding the rebuild order); marimo's caching is a content-hash rebuilder paired with a reactive scheduler, with no persistent build trace. We borrow the recursive-hash derivation lookup from Nix [20], [21] and apply it to a reactive notebook instead of a static package graph — closer to Nix's input-addressed model than to Shake's verifying traces. The data-engineering analog is Bauplan/Nessie's pipeline-stage hashing [22], and workflow engines persist the same discipline at task granularity: Nextflow's `-resume` and Snakemake's `--cache` key results on code, parameters, and input hashes [23], [24]. Beneath the systems layer, self-adjusting computation and Adapton supply the change-propagation theory that DAG memoization specializes [25], [26].

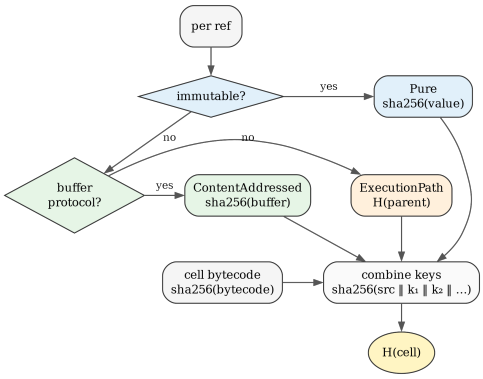
For existing scientific-Python memoization, mandala [12] is the closest analog at this venue (SciPy 2024); it memoizes calls inside a `with storage:` context using `joblib.hash` for content addressing and persists call provenance. signac [27] is an earlier parameter-hashed predecessor, diskcache [15] is the byte-keyed control where content addressing is the user's responsibility, and joblib's `Memory` [28], scientific Python's standard persistent memoizer, keys on pickled arguments per decorated function. Notebook-state work is complementary: Kishu [13] snapshots co-variable groups for session time travel, ElasticNotebook [14] migrates state live, and noWorkflow [29] traces execution into a queryable provenance DAG. Checkpoints store *state* where a cache stores *results*, and the two compose. Two concurrent systems bracket ours: NBRewind checkpoints notebook cells incrementally to reproduce distributed workflows [30], and Chordata formalizes shortcut memoization for live re-

execution at the interpreter level [31]. Neither derives keys from a reactive graph — the property the rest of this paper exploits.

3. Cache Key Construction

For computational caching, false positive hits are unacceptable and false negatives merely undesirable: key derivation must be sensitive to value changes yet robust to superficial notebook edits. A naive key would hash every ref's value directly. This does not survive Python's exposure of mutation and FFI — pointers move under realloc, weakrefs report identity rather than content, and opaque C-extension objects expose neither a buffer nor a stable repr — so value-only keys generate false negatives as process state shifts beneath an unchanged notebook. Hashing the cell's source alone fails in the other direction: cells that read the filesystem, network, or wall clock — and marimo's (heavily discouraged) in-place mutation — would be invisible to the key, generating false positives. Build systems face the same dilemma and substitute the producer when the artifact is opaque [20]; marimo's cache key follows the same discipline, derived recursively over the reactive DAG. Each cell's key depends on (a) the cell's compiled body — bytecode rather than source text, so comments and formatting do not participate — (b) a content address for every reference the cell reads, and (c) the keys of the parent cells that own those references. Some refs cannot be addressed directly (mutable state, side-effecting refs, large values without a typed buffer protocol). For those we substitute the parent-cell hash; when an input cannot be addressed directly, address the producer that defines it.

3.1. *Key dispatch*




```python
def derive_key(cell, graph, scope) -> HashKey:
    refs = ScopedVisitor(cell.module).refs          # static analysis
    refs = normalize_scope(refs, scope)             # UI/state -> values
    h, kind = sha256(), "Pure"                      # no refs: pure cell

    if refs:                                        # content-address first
        kind = "ContentAddressed"
        refs, serial = content_serialize(refs, scope)
        for ref in sorted(serial):
            h.update(serial[ref])                   # buffer-protocol view

    if refs:                                        # substitute producers
        kind = "ExecutionPath"
        for parent in producing_cells(refs, graph):
            h.update(derive_key(parent, graph, scope).digest)

    if refs - scoped_refs:                          # same-cell context
        kind = "ContextExecutionPath"
        h.update(hash_module(cell.context))

    h.update(hash_module(compile(cell.module)))     # bytecode, not source
    return HashKey(h.digest(), kind)
```

Figure 1. *Cache key construction. Left: the per-ref dispatch, a three-way fallback into Pure (hash the value), ContentAddressed (hash the buffer), or ExecutionPath (substitute the producing cell's H); the first match emits the per-ref key, and every per-ref key feeds one sha256 combiner with the cell's compiled-body hash to produce $H(c)$. Right: the derivation over the full cell, abridged from* `BlockHasher.__init__` *(*`marimo/_save/hash.py`*).*

The dispatch is a flat three-way fallback, and the listing beside it makes the recurrence over the full cell explicit (Figure 1). Pure cells reference nothing outside their own body; the key reduces to the hash of the compiled cell. ContentAddressed refs expose their bytes directly: Python primitives and frozen collections, values captured by marimo's intentional notebook state (UI elements and `mo.state`, hashed by current value), and buffer-protocol objects. An important case for scientific computing is the numpy ndarray, hashed from its contiguous buffer without serialization; other objects advertising numpy's array interface normalize to the same byte view — an idiom borrowed from joblib [28] and mandala [12]. ExecutionPath refs are not themselves directly hashable but are *cell-owned* — produced by an upstream cell with a known cache key — so we substitute the parent keys, recursively determined by the same dispatch. A fourth outcome in the listing, ContextExecutionPath,

covers references defined in the same cell as the cached block — the code preceding a `mo.persistent_cache` context manager — whose surrounding context is folded into the key. The decorators `@mo.cache` / `@mo.persistent_cache` apply the same dispatch to function call arguments, resolved at call time. The streamlit `cache_data` / `cache_resource` split [32] anticipates the data-vs-resource distinction that ContentAddressed and ExecutionPath resolve.

3.2. Parent-hash substitution

For each cell c we record a hash $H(c)$ at the end of the cell's execution. When a downstream cell reads a ref r produced by c and r is not directly addressable, we substitute $H(c)$ for r in the downstream key — the `ExecutionPath` branch of Figure 1. The downstream key then invalidates exactly when the producer's key does, which is exactly when the reactive scheduler would re-run the cell. We do not require every value to be hashable in itself, only that its producing cell be hashable. The substitution is a special case of Hughes's lazy memo function model [33], where equality is by stored location rather than by deep value.

The recurrence builds a Merkle DAG [34] over the notebook: each cell's hash commits to the hashes of its parents, so an edit to one cell invalidates exactly the subtree it dominates. It gives the rebuilder its $O(|\text{ changed subtree}|)$ rehash cost; we never re-derive a cell's hash if none of its inputs have moved.

3.3. Worked example

These properties are asserted, not assumed: the notebook that builds this paper re-derives them with marimo's hasher (`BlockHasher`) on every render — invariance to comments, formatting, reordering, and cell identifiers; invalidation on public-name renames and upstream value changes; fixed keys under idempotent reruns — with one measured boundary: attribute mutation on a non-addressable object leaves every key fixed, a false positive the cache cannot detect (Limitations and Discussion).

Figure 2 visualizes the recurrence on the four-cell DAG codified below (abridged; the exact sources ship in the notebook as `compute.WALKED_SOURCES`). A slider seeds a random input array, a small model class is constructed independently, and the forward pass binds them. `TinyNet` stands in for any opaque value a cell can produce — a torch `nn.Module`, a database connection, a compiled kernel. Each branch of the dispatch is exercised: `a` is Pure, `b` is ContentAddressed via the array's buffer, and `d` substitutes $H(c)$ for the unhashable model instance. `c` itself keys cheaply as ContentAddressed — its only outside reference is the module-pinned `np` — even though its *product* cannot be hashed; substituting the producer spares every consumer.

```
# a · seed          # b · random input              # c · model                # d · forward pass
seed = 7             x = rng(seed).normal(64)       model = TinyNet()          y = model(x)
```

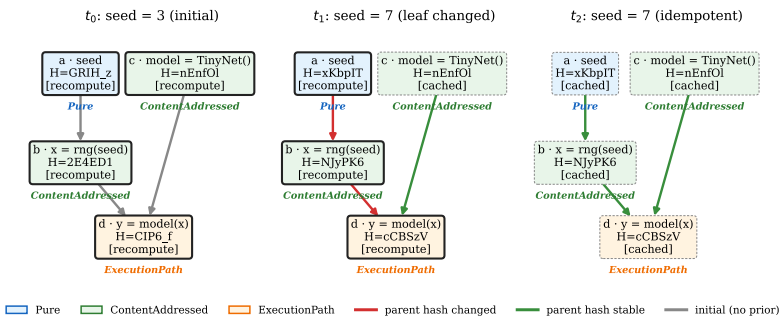


Figure 2. *Worked example of the recurrence on the four-cell DAG codified above. Every hash and branch label is emitted by marimo's hasher over a compiled cell graph at render time. Moving the seed ($t_0 \to t_1$) invalidates* `a`*,* `b`*, and* `d` *(red edges);* `c` *stays cached because the seed is not among its refs. Re-rendering with the same seed ($t_1 \to t_2$) leaves every hash fixed (green edges) and the rebuilder reuses every result. The italic label under each box names the dispatch branch the cell exercises.*

4. Storage and Loading

Cache keys identify values, but a notebook does not always need the value itself: a downstream cell may forward a reference without inspecting it, and a static export may serialize a value for a reader who never executes the producing cell. Storage and loading are therefore decoupled from lookup. Of marimo's two persistent backends, the default `PickleLoader` serializes the full `Cache` envelope as one pickle blob, re-materializing every def eagerly on lookup; the opt-in `LazyLoader` writes a JSON manifest of per-def references alongside individual blob files and hydrates values on demand through a stub mechanism that crosses process and session boundaries — the same protocol whether the value comes from local disk or a sidecar inside a static HTML bundle.

4.1. Store interface and LazyLoader

The store exposes a single `ReferenceStub` protocol with one `load()` method and codec-specific subclasses for pickle, joblib, numpy `.npy`, and Arrow; a lookup returns a stub, and deserialization happens on the stub's first access, not on lookup. The split charges deserialization to the consumer that actually reads the value — a downstream cell may bind a stub by name without forcing it — and a single stub kind chooses its codec from the value type at save time.

4.2. WASM portability

marimo exports notebooks to static HTML through a Pyodide WASM bundle. Because the cache is content addressed and the loader is portable, cached values participate in that export: `--execute` runs the notebook once and bundles the resulting manifests and blobs, and a reader's browser re-derives the same keys and rehydrates each value on first access, exactly as in an interactive session. Scientific articles (such as this one), general-audience blog posts, and educational materials all benefit, shipping heavy results inside the notebook itself. Two parity constraints follow: the export runs under Pyodide's CPython release, since bytecode hashes are minor-version specific, and module versions stay out of the key (`pin_modules=False`), since host and browser environments never coincide. Manifests are Ed25519-signed at export and verified before rehydration; a failed blob is treated as a miss, so tampered data is never silently served.

The export ships values rather than the code that produced them, so a cell may depend on packages the browser cannot import, as long as the values it defines serialize to a portable codec. As a demonstration (`demo/` in this paper's artifact), the export host trains a PyTorch model, exports it to ONNX bytes behind a small runtime wrapper, and slices a prediction sweep into numpy arrays. In the browser, where `import torch` is impossible, the arrays and the wrapper restore through their codecs — a custom stub rebinds the ONNX bytes to `onnxruntime-web` — and a slider drives live in-browser inference. Values that resist serialization restore as named stubs that raise on first access, so partial restoration stays safe. The benchmark data behind Figure 3 ships through the same export as a kilobyte-scale sidecar, because a cell's cache stores its defs — the measurement rows — while the multi-hundred-megabyte payloads they timed never leave the bench.

5. Evaluation

Caching is a conditional win: the cache pays key derivation plus value load on a hit, and key derivation plus value save on a miss. When the goal is portability and provenance the time penalty is inconsequential; to skip recomputation, the cache must pay back its overhead by avoiding a more expensive cell body. We validate the implementation by measuring three cost components (key derivation, value load, value save) separately and as end-to-end hit and miss paths on numpy `float64` payloads from 1 MB to 500 MB, bracketed by representative baselines: mandala's decorated-function memoization and diskcache's byte-keyed store (the no-derivation floor). The baselines situate the overhead; they solve adjacent problems, not this one, and the contribution remains the derived key and what it unlocks.

5.1. End-to-end cache evaluation

What notebook users observe is not per-call lookup latency but the wall time from "edit upstream" to "downstream value bound in Python." Panel (a) of Figure 3 reports that composite metric across six strategies. All cluster up to roughly 50 MB. Above that, the spread is host-dependent: where disk dominates, strategies split along the in-memory / persistent boundary; where key derivation dominates, the persistent variants ride the same curve as the in-memory `mo.cache` bound. The 100 ms dashed line marks the threshold below which a response reads as instantaneous [35] — worth defending, since added latency measurably suppresses exploration [36] — and every persistent method holds it across typical exploratory payloads.

The dotted curve in panel (a) prices the miss path. With hit rate p, caching pays when the cell body costs more than $T_{hit} + \frac{1-p}{p}\left(T_{key} + T_{save}\right)$. Because the measured miss overhead is comparable to the hit cost on these payloads, the break-even body cost at $p = 0.9$ rides roughly 10% above the hit curve.

Panel (b) decomposes the largest-payload hit into key derivation and value load, alongside the overhead a miss adds (key derivation and value save). marimo and mandala both content-address the value; they differ in the primitive. mandala derives its key via `joblib.hash`, which serializes through pickle before hashing [12]; marimo's `data_to_buffer` views the ndarray's contiguous bytes through Python's buffer protocol and hashes them directly. How much the pickle pass costs is strongly host-dependent: on the Apple M2 Pro used for the camera-ready figures, hashing is fast and the pickle pass costs mandala roughly 3× end to end; on a Linux x86-64 server, memory bandwidth makes the pickle pass nearly free, value load dominates instead, and the penalty compresses to roughly 1.2×. The panel-(a) ordering — marimo at or below mandala at every payload — held on all three hosts measured (Apple M2 Pro, Linux server, Linux workstation); the figure carries its build host and ratio so any reproduction shows its own number. marimo's value load tracks the `diskcache (fixed key)` floor, isolating the structural gap in key derivation rather than storage. Above the primitive, the differentiator is ergonomic: mandala caches at decorated-function boundaries inside `with storage:` blocks, while marimo caches at cell boundaries derived from the reactive DAG. The byte-keyed alternatives bracket the comparison: a pre-built byte key is the fastest possible lookup but cannot detect input changes, and `diskcache.memoize` pays pickle on both the key and its SQLite blob store. Panel (c) exposes per-call variance: irregular fsyncs and page-cache turnover surface as a long tail, longest for the lazy variant's background-writer settle.

The key path generalizes beyond contiguous ndarrays: containers of arrays and Arrow tables normalize to byte views at comparable cost; a large homogeneous Python list pays an element-wise walk roughly an order of magnitude slower — numeric data belongs in arrays before it crosses a cached boundary. Framework-native formats slot into the same split: torch tensors already key through the buffer protocol, and a `.pt` codec in the lazy

registry round-trips them 1.4–1.7× faster than the joblib pairing at 50 and 500 MB (`lib/bench_torch.py` in this paper's artifact).

The sweep behind Figure 3 is itself served by the mechanism under test: `@mo.persistent_cache` wraps every measurement function, keyed additionally on a host fingerprint and a bench-version string; the sweep cost 91 s cold on the Linux server build, re-binds in under a millisecond warm, and editing any measurement function invalidates exactly the affected sweeps.

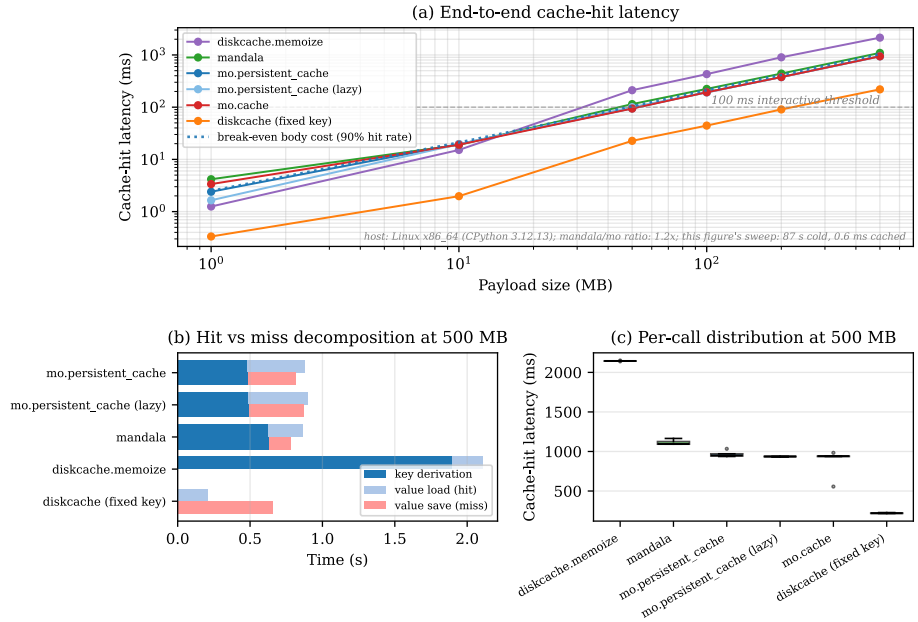


Figure 3. *End-to-end cache evaluation on numpy `float64` payloads. (a) Cache-hit latency vs payload size, log-log; the 100 ms dashed line marks the interactive threshold [35], the dotted curve the break-even body cost at a 90% hit rate. (b) Hit (key derivation + value load) and miss (key derivation + value save) decomposition at the largest sweep size, on the real disk-backed paths. (c) Per-method distribution of cache-hit samples at the largest size; `diskcache.memoize` fails outright past its SQLite blob ceiling, and failed sizes drop out of the plot. The host label carries the build host, the mandala/marimo ratio, and the cold-vs-cached cost of the figure's own sweep.*

The camera-ready figures are built on a MacBook Pro with an Apple M2 Pro; the full sweep also ran on the two Linux hosts as a consistency check, not a portability study, with the decomposition shifting between key-derivation-dominant and load-dominant as described above. The stage decomposition times the real disk-backed paths on both sides — `load_cache` for marimo, `joblib.load` on a temp-file blob for mandala — including the page-cache and envelope costs a primitive `pickle.loads` skips. All marimo timings carry the fix for a hash-memo bug (marimo PR #8805) that otherwise leaves decorator-path hits unrealistically flat.

6. Limitations and Discussion

We lead with limitations. The cache does not round-trip cells whose body closes over a non-serializable handle such as an open socket, CUDA device, or captured callable; rather than corrupt downstream state, the system bails to recomputation. Hash-memo flushing is coarse: a lifecycle event on a defining cell clears the whole memo, costing a redundant rehash on next access. Renaming is the false-negative class: public renames invalidate consumers by design, and renaming a `@mo.persistent_cache` function orphans its on-disk namespace. Library versions do not enter the key unless `pin_modules=True`, so an environment upgrade can serve stale hits; the portable WASM export must accept this gap, since

pinning would bind keys to the export host. Mutable refs that bypass the DAG — alias mutation through a closure, attribute writes on a non-addressable object — can still poison downstream cells; the invariance battery measures exactly this undetectable false positive (Rex [18] probes the same boundary). The lazy codec registry is young: pandas and polars frames take the Arrow path, while a raw `pyarrow.Table` still falls back to pickle. Signed manifests bind blobs to the exporting author, but signing attests provenance, not correctness: the reader still trusts that the author cached what the code computes.

Side effects — file reads, network calls, wall-clock queries — fold back into the key through hashable side-effect handles whose results contribute to the cell-level hash as if they were external references. Two constructors are currently provided, `mo.watch.file` and `mo.watch.directory`, keyed on content and listing; randomness or wall-clock time could bind to similar lifetime-managed handles. Outside that surface, side effects go unseen by the key.

7. Future work

Three lines follow from the measured boundaries. *Cost-aware policy*: with the write path measured, the break-even rule can run online — refuse to cache bodies cheaper than their own overhead, and track realized savings per cell. *Key scope*: pinning module versions by default would close the stale-hit gap. *Richer codecs*: landing the measured `.pt` tensor codec and a `pyarrow.Table` Arrow path, plus persistent cell-result reuse for agentic workflows driving long-running sessions [37]. Storage policy (eviction, per-codec footprint) and provenance-aligned cross-session memoization [29] remain open.

8. Conclusion

Given marimo's reactive DAG, content-addressed caching of cell results is a systems consequence rather than a novelty: the same parse that schedules a cell yields its cache key. Every render of this paper re-derives the invariance battery and the worked example with marimo's hasher, and the benchmarks are served by the cache they evaluate. On the payloads measured, hits track the byte-keyed floor within a small factor, and break-even sits roughly 10% above hit latency at a 90% hit rate. Cached artifacts ride the WASM export to readers with only a browser, a torch-trained model restoring as a live ONNX session in a page that cannot import torch. The keys that schedule a notebook thus carry its results across sessions, processes, and machines.

References

[1] J. F. Pimentel, L. Murta, V. Braganholo, and J. Freire, "A Large-Scale Study About Quality and Reproducibility of Jupyter Notebooks," in *Proceedings of the 16th International Conference on Mining Software Repositories (MSR '19)*, 2019, pp. 507–517. doi: 10.1109/MSR.2019.00077.

[2] F. van der Plas and Pluto.jl contributors, "Pluto.jl: Simple Reactive Notebooks for Julia." [Online]. Available: https://github.com/fonsp/Pluto.jl

[3] M. Bostock, "A Better Way to Code." [Online]. Available: https://medium.com/@mbostock/a-better-way-to-code-2b1d2876a3a0

[4] J. Valim and Livebook Team, "Livebook: Interactive and Collaborative Code Notebooks for Elixir." [Online]. Available: https://livebook.dev/

[5] S. Macke, H. Gong, D. J.-L. Lee, A. Head, D. Xin, and A. Parameswaran, "Fine-Grained Lineage for Safer Notebook Interactions," *Proceedings of the VLDB Endowment*, vol. 14, no. 6, pp. 1093–1101, 2021, doi: 10.14778/3447689.3447712.

[6] S. Macke, "IPyflow: A Next-Generation, Dataflow-Aware IPython Kernel." [Online]. Available: https://github.com/ipyflow/ipyflow

[7] B. Victor, "Inventing on Principle." [Online]. Available: https://worrydream.com/InventingOnPrinciple/

[8] A. Agrawal and M. Scolnick, "marimo: An Open-Source Reactive Notebook for Python." [Online]. Available: https://marimo.io/

[9] P. J. Guo and D. Engler, "Using Automatic Persistent Memoization to Facilitate Data Analysis Scripting," in *Proceedings of the 2011 International Symposium on Software Testing and Analysis (ISSTA '11)*, Toronto, ON, Canada, 2011, pp. 287–297. doi: 10.1145/2001420.2001455.

[10] Y. Xie, *Dynamic Documents with R and knitr*, 2nd ed. Boca Raton, FL: Chapman & Hall/CRC, 2015.

[11] Executable Books Project, "jupyter-cache: A defined interface for working with a cache of executed Jupyter notebooks." [Online]. Available: https://github.com/executablebooks/jupyter-cache

[12] A. Makelov, "Mandala: Compositional Memoization for Simple & Powerful Scientific Data Management," in *Proceedings of the 23rd Python in Science Conference (SciPy 2024)*, 2024. doi: 10.25080/JHPV7385.

[13] Z. Li, S. Chockchowwat, R. Sahu, A. Sheth, and Y. Park, "Kishu: Time-Traveling for Computational Notebooks," *Proceedings of the VLDB Endowment*, vol. 18, no. 4, pp. 970–985, 2025, doi: 10.14778/3717755.3717759.

[14] Z. Li, P. Gor, R. Prabhu, H. Yu, Y. Mao, and Y. Park, "ElasticNotebook: Enabling Live Migration for Computational Notebooks," *Proceedings of the VLDB Endowment*, vol. 17, no. 2, pp. 119–133, 2024, doi: 10.14778/3626292.3626296.

[15] G. Jenks, "DiskCache: Disk and File Backed Cache for Python." [Online]. Available: https://github.com/grantjenks/python-diskcache

[16] D. Michie, "``Memo'' Functions and Machine Learning," *Nature*, vol. 218, no. 5136, pp. 19–22, 1968, doi: 10.1038/218019a0.

[17] D. Koop and J. Patel, "Dataflow Notebooks: Encoding and Tracking Dependencies of Cells," in *9th USENIX Workshop on the Theory and Practice of Provenance (TaPP 2017)*, Seattle, WA, 2017. [Online]. Available: https://www.usenix.org/conference/tapp17/workshop-program/presentation/koop

[18] M. Zheng, W. Crichton, A. Narayan, D. Raghavan, and N. Vasilakis, "When Are Reactive Notebooks Not Reactive?." [Online]. Available: https://arxiv.org/abs/2511.21994

[19] A. Mokhov, N. Mitchell, and S. Peyton Jones, "Build Systems à la Carte," *Proceedings of the ACM on Programming Languages*, vol. 2, pp. 1–29, 2018, doi: 10.1145/3236774.

[20] E. Dolstra, M. de Jonge, and E. Visser, "Nix: A Safe and Policy-Free System for Software Deployment," in *Proceedings of the 18th USENIX Conference on System Administration (LISA '04)*, Atlanta, GA, 2004, pp. 79–92. [Online]. Available: https://www.usenix.org/legacy/event/lisa04/tech/dolstra.html

[21] E. Dolstra, "The Purely Functional Software Deployment Model." [Online]. Available: https://edolstra.github.io/pubs/phd-thesis.pdf

[22] C. Greco and J. Tagliabue, "Reproducible Data Science over Data Lakes: Replayable Data Pipelines with Bauplan and Nessie," in *Proceedings of the Eighth Workshop on Data Management for End-to-End Machine Learning (DEEM@SIGMOD '24)*, 2024. doi: 10.1145/3650203.3663336.

[23] P. Di Tommaso, M. Chatzou, E. W. Floden, P. P. Barja, E. Palumbo, and C. Notredame, "Nextflow enables reproducible computational workflows," *Nature Biotechnology*, vol. 35, no. 4, pp. 316–319, 2017, doi: 10.1038/nbt.3820.

[24] F. Mölder *et al.*, "Sustainable data analysis with Snakemake," *F1000Research*, vol. 10, p. 33, 2021, doi: 10.12688/f1000research.29032.2.

[25] U. A. Acar, G. E. Blelloch, and R. Harper, "Adaptive Functional Programming," in *Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '02)*, Portland, OR, USA, 2002, pp. 247–259. doi: 10.1145/503272.503296.

[26] M. A. Hammer, Y. P. Khoo, M. Hicks, and J. S. Foster, "Adapton: Composable, Demand-Driven Incremental Computation," in *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '14)*, Edinburgh, United Kingdom, 2014, pp. 156–166. doi: 10.1145/2594291.2594324.

[27] V. Ramasubramani, C. S. Adorf, P. M. Dodd, B. D. Dice, and S. C. Glotzer, "signac: A Python Framework for Data and Workflow Management," in *Proceedings of the 17th Python in Science Conference (SciPy 2018)*, 2018, pp. 152–159. doi: 10.25080/Majora-4af1f417-016.

[28] Joblib Developers, "Joblib: Running Python Functions as Pipeline Jobs." [Online]. Available: https://joblib.readthedocs.io/

[29] J. F. Pimentel, L. Murta, V. Braganholo, and J. Freire, "noWorkflow: A Tool for Collecting, Analyzing, and Managing Provenance from Python Scripts," *Proceedings of the VLDB Endowment*, vol. 10, no. 12, pp. 1841–1844, 2017, doi: 10.14778/3137765.3137789.

[30] T. Azaz, R. Ahmad, M. S. Islam, D. Thain, and T. Malik, "Efficiently Reproducing Distributed Workflows in Notebook-based Systems." 2026.

[31] M. Kirisame *et al.*, "Incremental Live Programming via Shortcut Memoization." 2026.

[32] Streamlit Team, "Introducing Two New Caching Commands to Replace st.cache." [Online]. Available: https://blog.streamlit.io/introducing-two-new-caching-commands-to-replace-st-cache/

[33] R. J. M. Hughes, "Lazy Memo-Functions," in *Proceedings of the Conference on Functional Programming Languages and Computer Architecture (FPCA '85)*, in LNCS, vol. 201. 1985, pp. 129–146. doi: 10.1007/3-540-15975-4_34.

[34] R. C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," in *Advances in Cryptology — CRYPTO '87*, in Lecture Notes in Computer Science, vol. 293. 1988, pp. 369–378. doi: 10.1007/3-540-48184-2_32.

[35] S. K. Card, G. G. Robertson, and J. D. Mackinlay, "The Information Visualizer, an Information Workspace," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '91)*, New Orleans, LA, USA, 1991, pp. 181–186. doi: 10.1145/108844.108874.

[36] Z. Liu and J. Heer, "The Effects of Interactive Latency on Exploratory Visual Analysis," *IEEE Transactions on Visualization and Computer Graphics*, vol. 20, no. 12, pp. 2122–2131, 2014, doi: 10.1109/TVCG.2014.2346452.

[37] T. Manz, M. Scolnick, and A. Agrawal, "Beyond the Shell: Extending Agents with Reactive Python Notebooks," in *Proceedings of the SAO Workshop on AI Agents and Data Systems (CAIS 2026)*, San Jose, CA, USA, 2026. [Online]. Available: https://bauplanlabs.github.io/SAO-workshop/papers/45.pdf